Weak Randomness in Android Session Token Generation: A Static Analysis Case Study

Jiakai Bao
University of Sydney

Clarissa Mo
University of Sydney

Yi Qiao
University of Sydney

Jiazhou Shen
University of Sydney

Lina He
University of Sydney

Handong Ding
University of Sydney

Abstract

We present a security analysis of an Android application (MASTG_TEST0016) focusing on cryptographic vulnerabilities related to randomness. Through static analysis using JADX and APKTool, we identified a critical vulnerability where session tokens are generated using `java.util.Random` instead of `java.security.SecureRandom`. This allows attackers to predict session tokens and potentially hijack user sessions. We provide a detailed threat model, attack scenario, and a concrete mitigation strategy.

1 Introduction

Android application security analysis requires decompiling APK files to inspect source code. An APK (Android Package) is the installable package format containing compiled code, resources, and manifest. Decompilation transforms Dalvik bytecode back to readable Java/smali code, enabling security audits.

This paper analyzes the MASTG_TEST0016 application to identify vulnerabilities related to randomness and cryptographic misuse. We employed JADX for Java decompilation and APKTool for resource extraction. Our analysis discovered that the application uses a weak pseudo-random number generator (PRNG) for generating security-sensitive session tokens.

2 System and Threat Model

2.1 Application Overview

The target application is a simple user registration and login system with three components:

- **MainActivity**: User registration - stores credentials locally
- **Login**: Authentication - validates credentials, creates session
- **Profile**: User profile (blank placeholder) - protected by session token

2.2 System Model

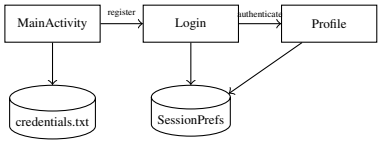

Figure 1: Application Data Flow

2.3 Assets and Threat Model

Protected Assets:

- User credentials (username/password)
- Session tokens (authentication state)
- User profile page

Attacker Model: We consider a local attacker who can:

- Install and run the target application
- Register accounts and trigger login operations
- Access SharedPreferences on rooted devices
- Observe generated session tokens from their own sessions

Attacker Goal: Predict legitimate users' session tokens to bypass authentication and gain unauthorized access to their profiles.

3 Vulnerability Discovery

3.1 Methodology

We used the following AI-assisted workflow:

1. Decompiled APK using JADX (Java) and APKTool (smali/resources)
2. Extracted package info from AndroidManifest.xml
3. Used AI to identify code patterns related to randomness: `Random`, `token`, `session`
4. Analyzed smali bytecode to confirm vulnerability

3.2 Vulnerability Location

File: `com/example/mastg_test0016/Login.java` and `res/AndroidManifest.xml`

Methods: (Lines 111-143) `generateSessionToken()`: generates a 16-character token using `Random.nextInt(62)`; `createSession()`: stores the token as `sessionToken` in `SessionPrefs`.

```
    public void createSession() {
        this.editor.putString(KEY_SESSION_TOKEN, generateSessionToken());
        this.editor.apply();
    }
    public String getSessionToken() {
        return this.sharedPreferences.getString(KEY_SESSION_TOKEN, null);
    }
    private String generateSessionToken() {
        Random random = new Random();
        StringBuilder sb = new StringBuilder(16);
        for (int i = 0; i < 16; i++) {
            sb.append("
                    ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789
                    ".charAt(random.nextInt(62)));
        }
        return sb.toString();
    }
}
```

4 Vulnerability Analysis

4.1 Technical Details

The vulnerability stems from using `java.util.Random` instead of `java.security.SecureRandom` [1]:

Property	java.util.Random	SecureRandom
Algorithm	Linear Congruential	Cryptographic PRNG
Brute-force time	Days (feasible)	Infeasible
Predictability	Predictable	Unpredictable
Seed	System.nanoTime()	OS entropy pool
Seed Space	48 bits	128+ bits
CWE-338 satisfied	Yes	No

4.2 Attack Scenario

1. **Data Collection:** The attacker registers and logs in multiple times, collecting session tokens $T_1, T_2, ..., T_n$ generated by the same pseudo-random number generator.
2. **State Analysis:** Since `java.util.Random` is a deterministic linear congruential generator with a 48-bit internal state, its outputs are theoretically predictable given sufficient observations.
3. **Token Prediction:** From observed outputs, the attacker may infer the PRNG state and predict future tokens such as T_{n+1}.
4. **Session Manipulation:** If the application relies solely on the session token to represent authenticated state, a predicted token may be reused or injected to impersonate a user.
5. **Authentication Implication:** Session tokens are generated after successful login and thus serve as proof of authentication. Their predictability weakens access control and may enable unauthorized access.

Exploitation Complexity: Each token leaks partial PRNG information (6 bits/character). Despite this, the 48-bit state makes prediction feasible via inference or brute-force, posing a practical risk of authentication bypass.

5 Mitigation

Replace the weak PRNG with a cryptographically secure alternative:

```
private String generateSessionToken() {
    SecureRandom sr = new SecureRandom();
    StringBuilder sb = new StringBuilder(32);
    String chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ" +
        "abcdefghijklmnopqrstuvwxyz0123456789";
    for (int i = 0; i < 32; i++) {
        sb.append(chars.charAt(
            sr.nextInt(chars.length())));
    }
    return sb.toString();
}
```

Key Changes:

- `SecureRandom` provides cryptographic randomness
- Token length increased from 16 to 32 characters (192 bits entropy)

6 Conclusion

We identified a critical cryptographic vulnerability in an Android application where session tokens are generated using a predictable PRNG. This violates fundamental security principles and enables session hijacking attacks. The fix is straightforward: replace `java.util.Random` with `SecureRandom`. This case study demonstrates the importance of static analysis in mobile application security assessments.

References

[1] ORACLE. Class securerandom (java se 17). https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/security/SecureRandom.html, 2024. Accessed: 2026-03-29.